From Terraform and Terragrunt to Crossplane
A Gradual Migration at Scale



What is Crossplane

Create and manage (cloud) resources with kubectl

MR - Manage Resource is Kubernetes object declares and reconciles one external resource

instance.yaml

```yaml
apiVersion: ec2.aws.upbound.io/v1beta1
kind: Instance
metadata:
  annotations:
    crossplane.io/external-name: i-0b573*****
spec:
  forProvider:
    ami: ami-0123456789abcdef
    instanceType: t3.small
  deletionPolicy: Delete  # or Orphan
  managementPolicies:
    - Create
    - Observe
    - Update
    - Delete
```

external-name

Create: let the Provider record the EC2 ID.

Import: set the annotation to the exact existing ID, e.g. i-0abc…

– removes the …, the cloud resource running intact.

managementPolicies

This explicit list grants Create, Observe, Update and Delete. Remove Delete to keep AWS alive when the MR is removed.

What is a Provider

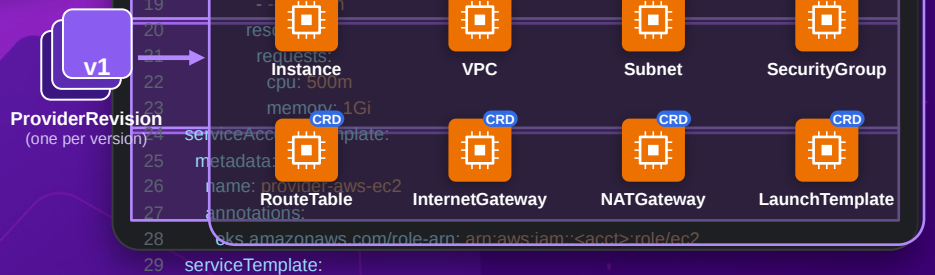
A Crossplane package that adds new Kubernetes APIs (CRDs) for an external service and runs the controller that reconciles them against that service's API.

provider.yaml

```yaml
apiVersion: pkg.crossplane.io/v1
kind: Provider
metadata:
  ...
package: xpkg.upbou...
packagePullPolicy: ...
revisionActivationPolicy: Automatic
revisionHistoryLimit: 1
packagePullSecrets:
  - name: registry-cre...
skipDependencyResolution: false
ignoreCrossplaneConstraints: false
runtimeConfigRef:
  name: aws-ec2-runtime
```

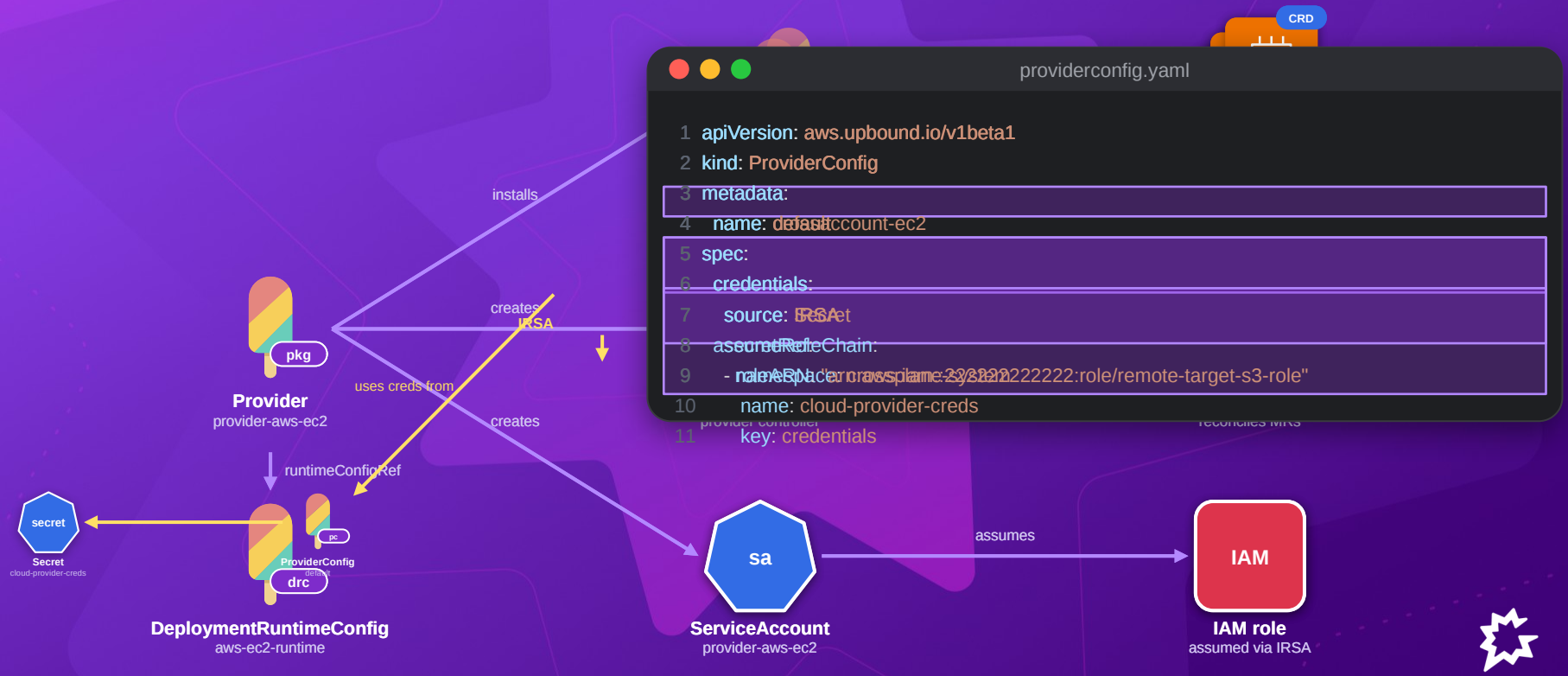
runtimeConfigRef

Points to the DeploymentRuntime Config that shapes the provider's pod. Default: **default** – Crossplane always makes sure it exists.



Provider at runtime

What installing a Provider creates – and how it gets AWS credentials (ProviderConfig)



Provider package

Who builds the package you install? Upbound publishes it · Upjet generates it

provider.yaml

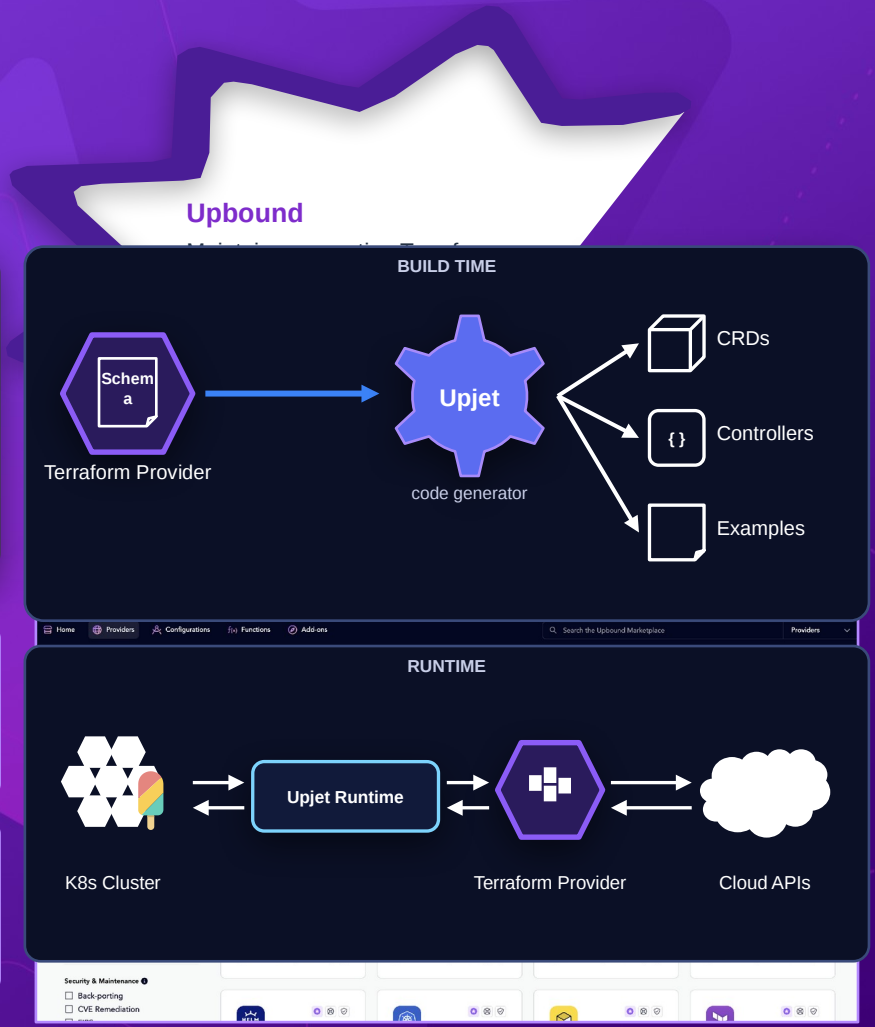
```yaml
apiVersion: pkg.crossplane.io/v1
kind: Provider
metadata:
  name: provider-aws-ec2
spec:
  package: xpkg.upbound.io/upbound/provider-aws-ec2:<pinned-version>
  packagePullSecrets:
    - name: upbound-pull-secret
```

Upbound

Company behind Crossplane and publisher of official provider packages, tooling and commercial products. Crossplane OSS does not require an Upbound service.

Upjet

Open-source framework that generates Crossplane Provider APIs and controllers from Terraform Provider schemas / logic. Upjet generates the Provider; it is not the Provider installed in the cluster.



Connection Secrets & Finalizers

Connection data is optional; deletion protection is part of the controller lifecycle

WHEN A RESOURCE RETURNS CREDENTIALS OR CONNECTION DATA

Typical examples

RDS endpoint + username/password, generated access keys, or a service URL that a workload needs at runtime.

```
spec:
  writeConnectionSecretToRef:
    name: payments-db-connection
# Secret is written in the MR namespace
```

The Provider must actually publish connection details. An EC2 Instance usually does not need this field just to expose instanceID or IP status fields.

Treat the Secret as a workload credential: scope RBAC, avoid logging values, and define rotation / handoff behavior.

MR DELETION

1 · Delete MR
Kubernetes sets deletionTimestamp.

→

2 · Finalizer
MR stays Terminating while cleanup runs.

→

3 · Provider
Calls AWS Delete or releases ownership.

Delete allowed

Provider terminates EC2 → confirms it is gone → removes finalizer → Kubernetes removes the MR.

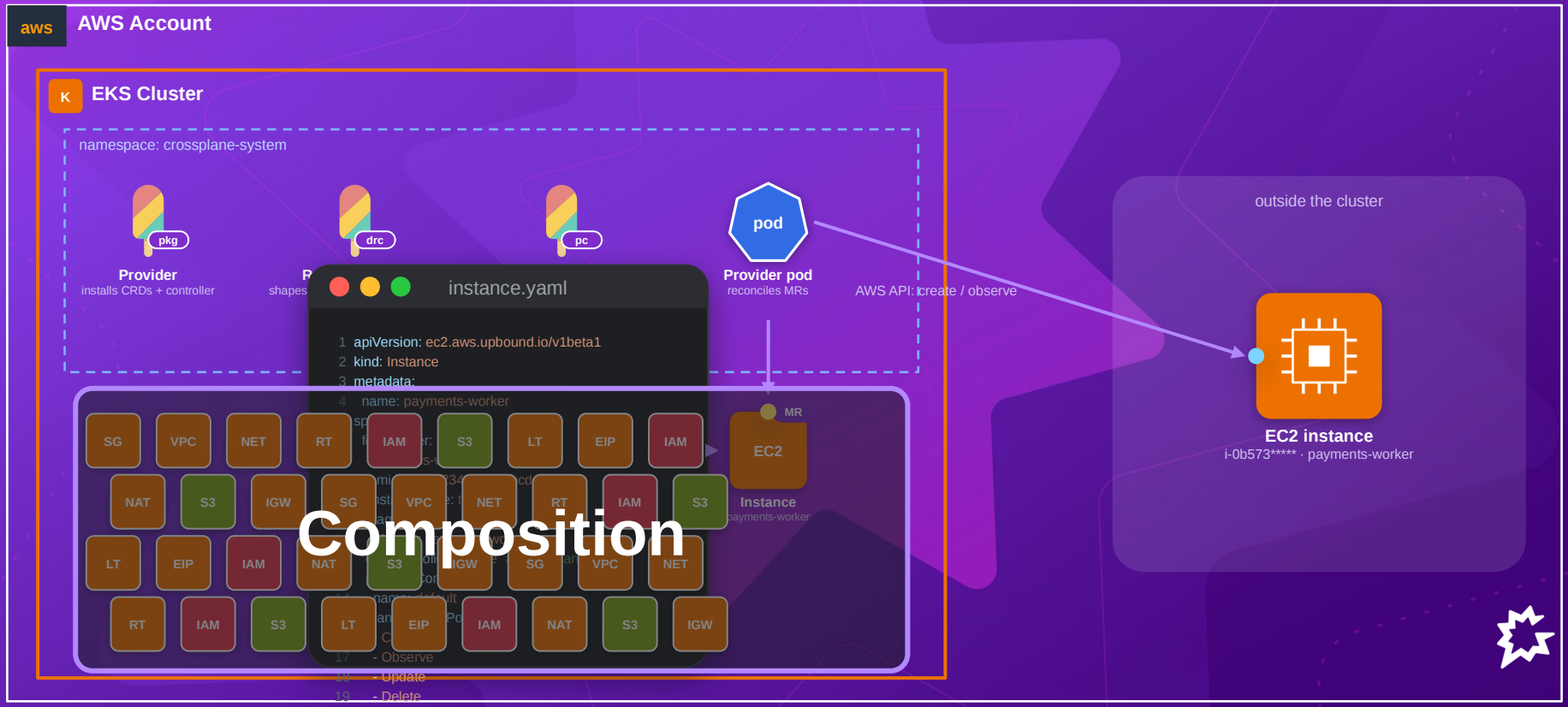
Delete not allowed / Orphan

Provider leaves AWS resource in place → clears its management relationship → finalizer is removed → MR disappears. Re-import requires a new MR and the exact external-name.

Finalizer = completion barrier for deletion; deletionPolicy / managementPolicies = what Crossplane is allowed to do to the cloud resource.

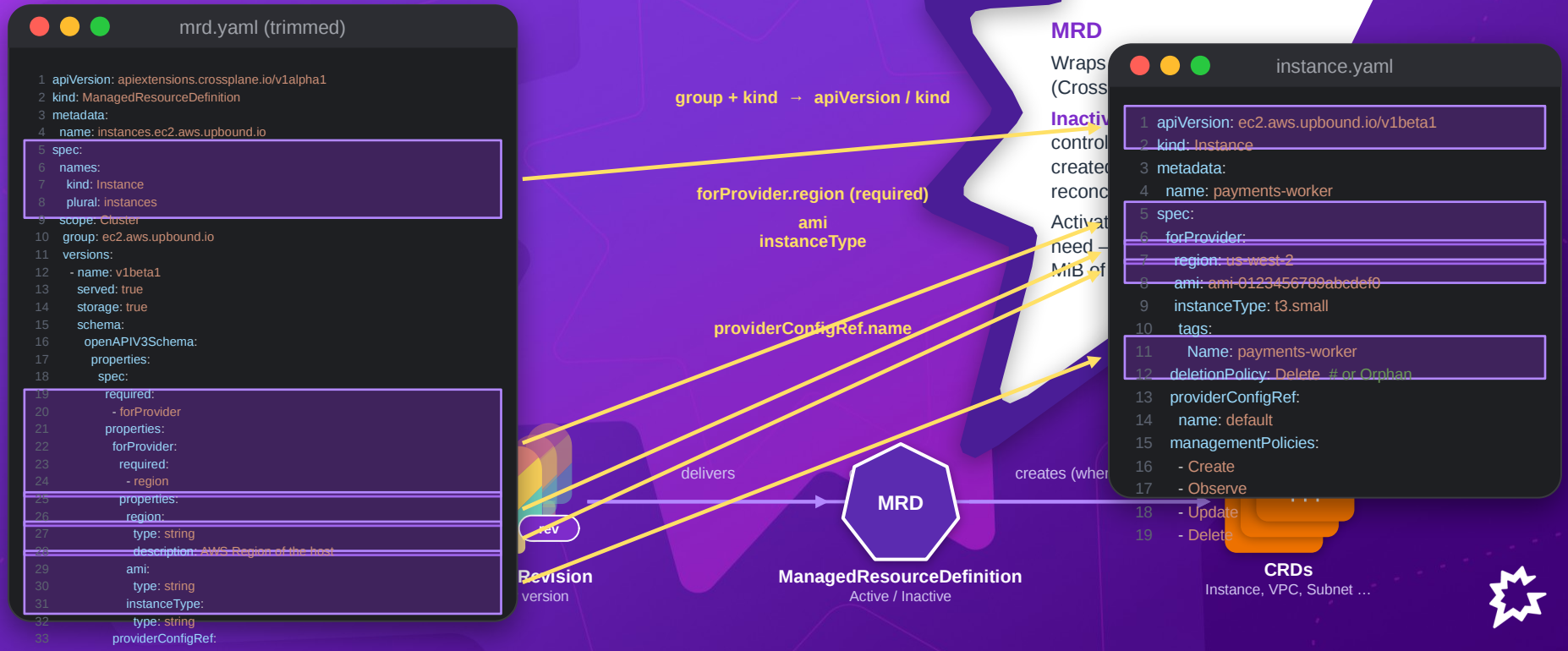
From YAML to one EC2 instance

What runs in the cluster, and how one MR becomes a real instance in the AWS account



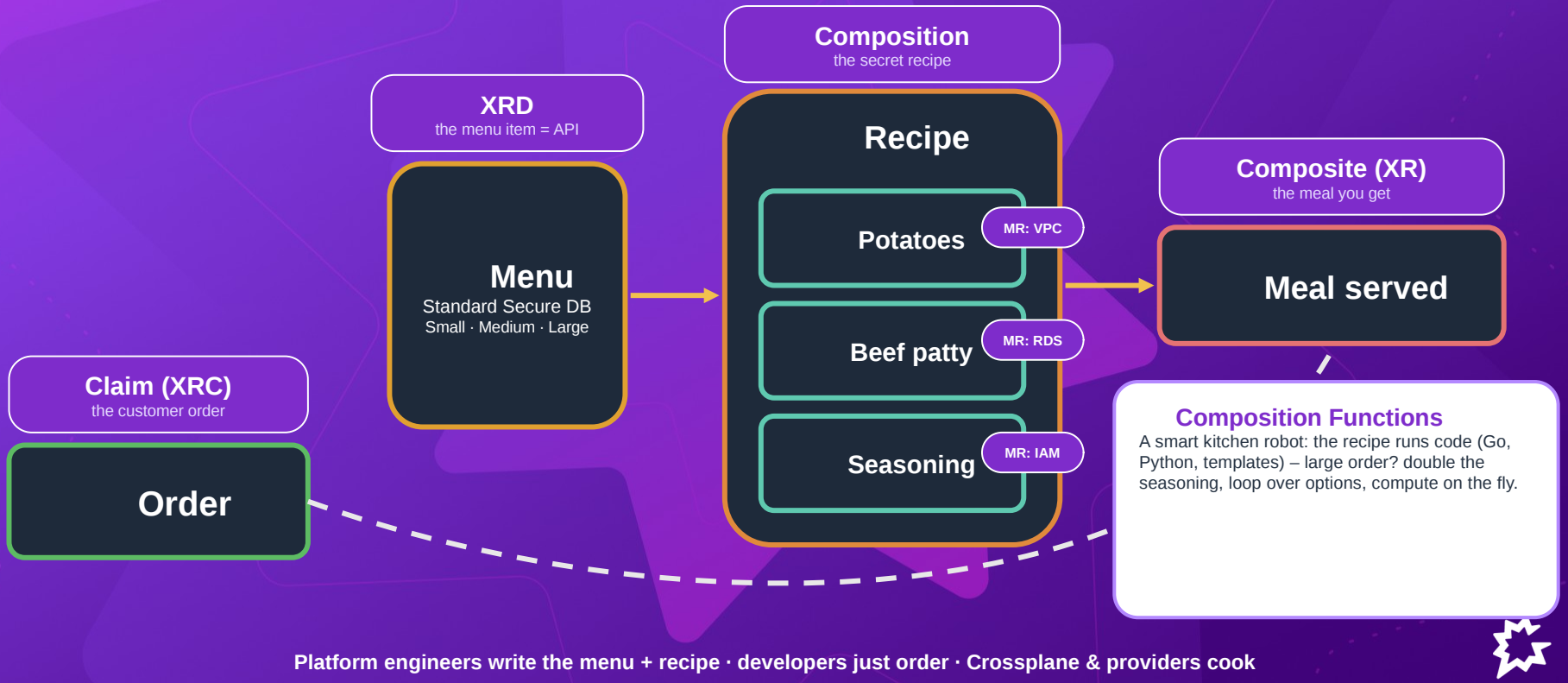
ManagedResourceDefinition (MRD)

Crossplane v2: the provider revision delivers MRDs – a CRD is created only for active ones



What is a Composition?

Developers order a meal – they don't buy raw potatoes, frozen patties and cooking oil



Platform engineers write the menu + recipe · developers just order · Crossplane & providers cook

XRD, XR & Claim

The XRD is the API contract; an XR or Claim is a request against that contract

xrd.yaml · the menu

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: CompositeResourceDefinition
metadata:
  name: xec2instances.compute.example.org
spec:
  group: compute.example.org
  names:
    kind: XEC2Instance
    plural: xec2instances
  claimNames:
    kind: EC2Instance
    plural: ec2instances
  versions:
  - name: v1alpha1
    served: true
    referenceable: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              parameters:
                type: object
                required: [region, ami, instanceType]
                properties:
                  region: {type: string}
                  ami: {type: string}
                  instanceType: {type: string}
```

composition.yaml · the recipe

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
metadata:
  name: xec2instances-aws
spec:
  compositeTypeRef:
    apiVersion: compute.example.org/v1alpha1
    kind: XEC2Instance
  mode: Pipeline
  pipeline:
  - step: patch-and-transform
    functionRef:
      name: function-patch-and-transform
    input:
      apiVersion: pt.fn.crossplane.io/v1beta1
      kind: Resources
      resources:
        - name: instance
          base:
            apiVersion: ec2.aws.upbound.io/v1beta1
            kind: Instance
          patches:
            - fromFieldPath: spec.parameters.region
              toFieldPath: spec.forProvider.region
            - fromFieldPath: spec.parameters.ami
              toFieldPath: spec.forProvider.ami
            - fromFieldPath: spec.parameters.instanceType
              toFieldPath: spec.forProvider.instanceType
```

claim.yaml · the order

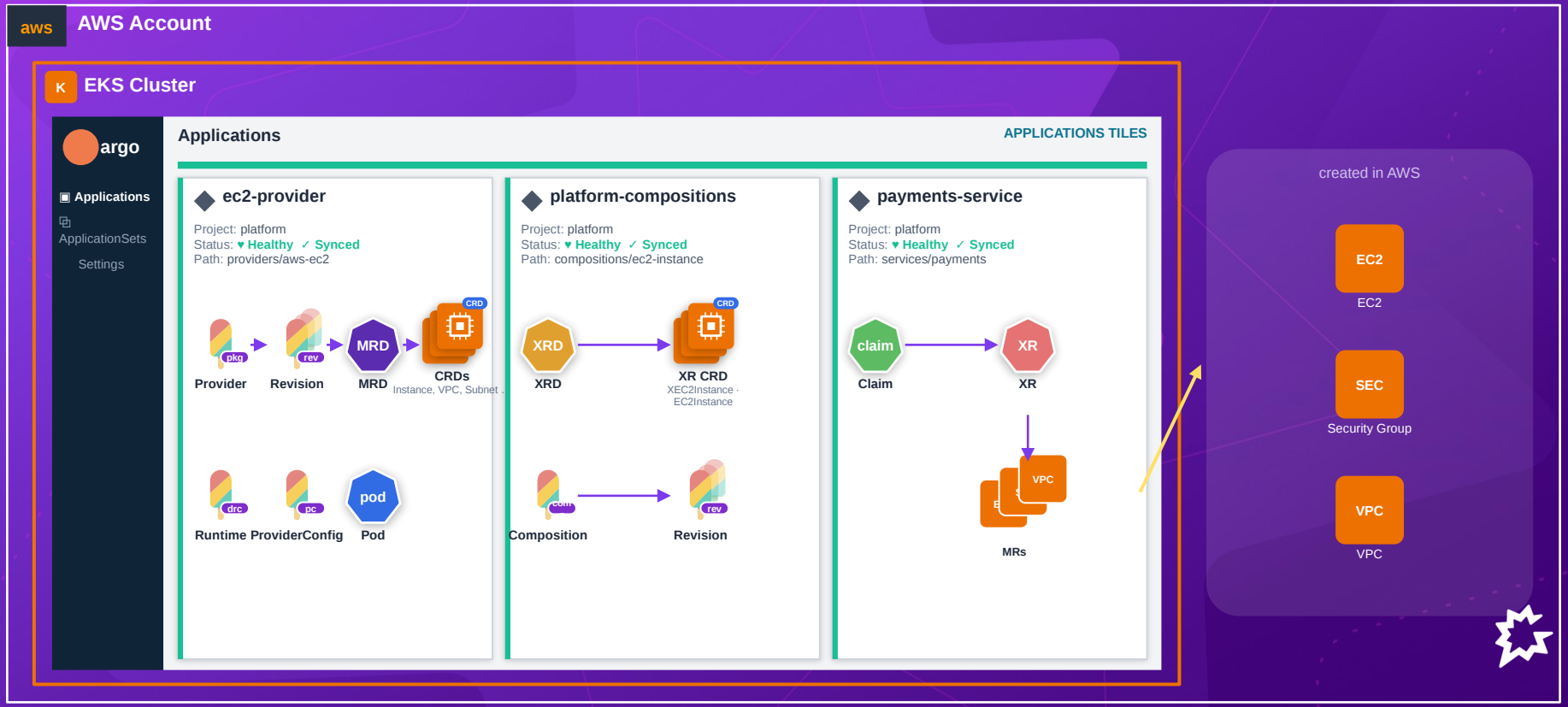
```yaml
apiVersion: compute.example.org/v1alpha1
kind: EC2Instance
metadata:
  name: payments-worker
  namespace: payments
spec:
  parameters:
    region: us-west-2
    ami: ami-0123456789abcdef0
    instanceType: t3.small
```

xr.yaml · the meal

```yaml
apiVersion: compute.example.org/v1alpha1
kind: XEC2Instance
metadata:
  name: payments-worker
spec:
  parameters:
    region: us-west-2
    ami: ami-0123456789abcdef0
    instanceType: t3.small
```

GitOps: Argo CD deploys it all

Three Argo CD apps: the provider, the platform API (XRD + Composition) and a service claim



Why Composition – not a Helm chart of MRs?

Both are GitOps-friendly. Composition adds an API, runtime wiring and one lifecycle.

	⎈ Helm chart of MRs	XRD + Composition
API contract	✗ values.yaml is free-form – typos and missing fields surface only when AWS rejects them.	✓ The XRD is a typed, validated API (required fields, enums, defaults) – kubectl explain works.
Wiring resources	✗ IDs (VPC → Subnet → SG) must be known when the chart is rendered.	✓ Values flow between resources at reconcile time (patches / functions read observed state).
One status	✗ Argo shows N separate MRs – you work out what is broken.	✓ One XR: Ready only when every composed resource is Ready.
Lifecycle	✗ Every MR lives on its own – pruning depends on the chart and Argo.	✓ Composed resources are owned by the XR – created, updated and deleted as one unit.
Safe rollout	✗ A new chart version changes every release on the next sync.	✓ CompositionRevisions – roll forward automatically or pin an XR to a revision (Manual update policy).
Guardrails	✗ Developers see raw IAM, tags and networking settings.	✓ The platform hides defaults (tags, IAM, naming) – developers fill a 5-line claim.

Composition Functions

Functions compute the desired composed resources; Providers reconcile those resources

COMPOSITION · COMPUTE-EC2 → XEC2INSTANCE · PIPELINE EXCERPT

```
spec:
  mode: Pipeline
  pipeline:
  - step: render-instance
    functionRef:
      name: function-patch-and-transform
    input:
      apiVersion: pt.fn.crossplane.io/v1beta1
      kind: Resources
      resources:
      - name: ec2
        base:
          apiVersion: ec2.aws.m.upbound.io/v1beta1
          kind: Instance
          spec:
            forProvider:
              region: us-west-2
              ami: ami-0123456789abcdef0
        patches:
        - type: FromCompositeFieldPath
          fromFieldPath: spec.parameters.instanceType
          toFieldPath: spec.forProvider.instanceType
  - step: readiness
    functionRef:
      name: function-auto-ready
```

WHAT A FUNCTION DOES

1 · Receives context

Observed XR, observed children, desired state so far and function input.

↓

2 · Computes desired state

Returns RunFunctionResponse; later steps can transform the output.

↓

3 · Crossplane applies MRs

Provider controllers, not the function, call AWS and reconcile EC2.

Use built-in functions for field mapping and standard readiness. Write a custom function for shared conditional logic; test with crossplane composition render.

Crossplane Composition Best Practices

What we learned composing real AWS infrastructure at Gong

in-cluster render | observe | dependencies | ArgoCD health | ready | crossplane render

Daniel Rubin · **DevOps, Gong**



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

COMPOSITION · MYAPP · RENDER STEP ONLY

```
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:         # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                  # template emits the MRs
      kind: GoTemplate
      inline:
        template: ...   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.

Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.

Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.

Hidden AWS errors

A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.

Challenge 1 · Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.

COMPOSITION



Composition

MyApp · checkout
creates the MRs →

All four MRs are created in the same reconcile

MANAGED RESOURCES (MRS)

S3	**Bucket** s3.aws.m.upbound.io/v1beta1	Ready ✓
IAM	**Role** iam.aws.m.upbound.io/v1beta1	Ready ✓
IAM	**Policy** iam.aws.m.upbound.io/v1beta1	Ready ✓
IAM	**RolePolicyAttachment** iam.aws.m.upbound.io/v1beta1	Ready ✓

…errors and retries with back-off until the Role and Policy exist

KUBECTL DESCRIBE ROLEPOLICYATTACHMENT

```
Warning
CannotResolveResourceReferences
cannot resolve references:
mg.Spec.ForProvider.PolicyArn: referenced field was
empty (referenced resource may not yet be ready)
```



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

COMPOSITION · MYAPP · RENDER STEP ONLY

```
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:         # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                  # template emits the MRs
      kind: GoTemplate
      inline:
        template: ...   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.

Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.

Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.

Hidden AWS errors

A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.

Challenge 2 · Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.

IN THE CLUSTER

S3

Bucket

checkout-data

Ready ✓

IAM

Policy

needs the bucket ARN

Synced: **True** · Healthy: **True**

Ready ✓

1 · CreateBucket

2 · status.atProvider.arn

aws

the ARN exists only after create

WHY SYNCED: FALSE? – THE RENDERED POLICY DOCUMENT

```
{ "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],
  "Resource": [
    "arn:aws:s3:::team-payments-checkout-data"
  ] }

# next reconcile – read from .observed.resources
```



What a Composition looks like

One render step creates every resource – but on its own it isn't enough

COMPOSITION · MYAPP · RENDER STEP ONLY

```
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:         # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                  # template emits the MRs
      kind: GoTemplate
      inline:
        template: ...   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.

Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.

Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.

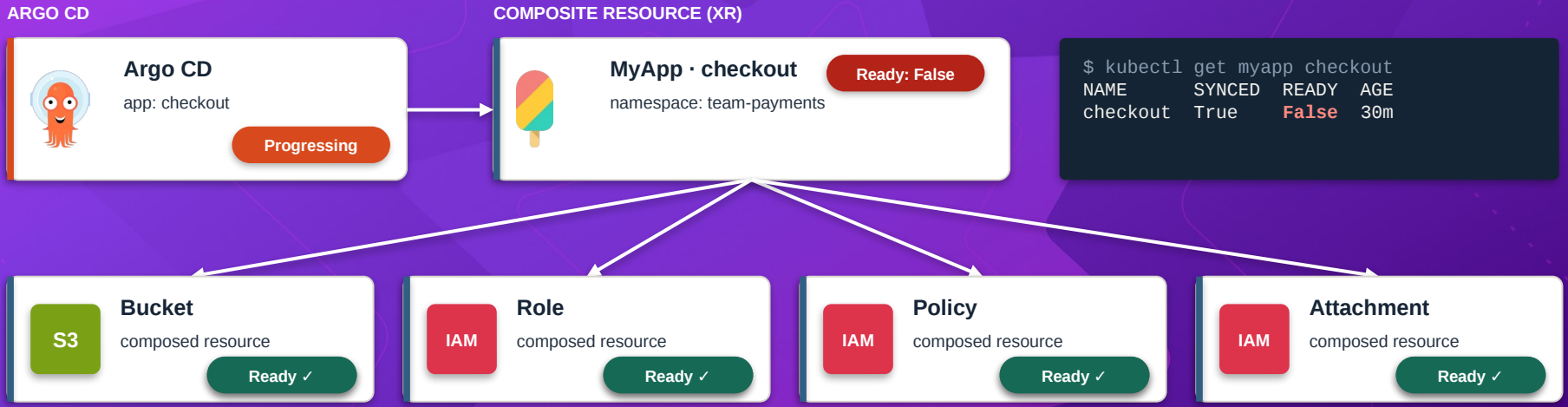
Hidden AWS errors

A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.

Challenge 3 · Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.



Pipeline mode: a function must mark each composed resource ready. None did → the XR is never Ready and Argo CD waits forever

What a Composition looks like

One render step creates every resource – but on its own it isn't enough

COMPOSITION · MYAPP · RENDER STEP ONLY

```
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
spec:
  compositeTypeRef:         # which XR it serves
    apiVersion: crossplane.best-practice.com/v1alpha1
    kind: MyApp
  mode: Pipeline
  pipeline:
  - step: render
    functionRef:
      name: function-go-templating
    input:                  # template emits the MRs
      kind: GoTemplate
      inline:
        template: ...   # Bucket, Role, Policy, Attachment
```

THE CHALLENGES

Wrong order

All MRs are applied at once – the attachment fails until Role and Policy exist.

Values that don't exist yet

The Policy needs the bucket ARN, which AWS returns only after create.

Never Ready

Nothing marks the composed resources Ready, so the XR stays not Ready.

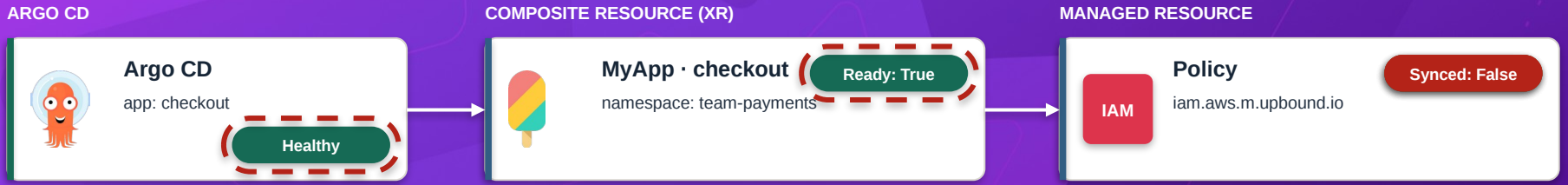
Hidden AWS errors

A failed update keeps the XR Ready – ArgoCD says Healthy.

Each challenge gets its own small function – next: the full MyApp pipeline.

Challenge 4 · Hidden AWS errors

A failed update keeps the XR Ready – ArgoCD says Healthy.



GIT PUSH – A SPEC CHANGE

```
spec:
  extraPolicyActions:
+   - s3:DeleteObject
```

KUBECTL GET POLICY CHECKOUT -O YAML

```
- type: Synced
  status: "False"
  message: update failed: AccessDenied: ... is not
    authorized to perform: iam:CreatePolicyVersion
```

The XR is still Ready → Argo CD still says Healthy. The real AWS error lives only on the child MR

Meet MyApp: our running example

One developer request → four functions → five AWS resources

THE ORDER · CROSSPLANE V2, NAMESPACED

```yaml
# what the developer writes
apiVersion: crossplane.best-
  practice.com/v1alpha1
kind: MyApp
metadata:
  name: checkout
  namespace: team-payments
spec:
  team: payments
  region: us-east-1
```

COMPOSITION PIPELINE · RUNS ON EVERY RECONCILE

1 · render

function-go-templating in-cluster render with observed state

2 · auto-ready

function-auto-ready child Ready → XR Ready

3 · sequence

function-sequencer hold resources until deps are Ready

4 · status

function-status-transformer lift child errors onto the XR

WHAT IT COMPOSES

Object

observe platform bucket

Observe

Bucket

app data

Role

Pod Identity trust

Policy

needs 2 ARNs from AWS

RolePolicyAttachment

after role + policy



Name your provider config default

Leave providerConfigRef out – the MR's CRD fills in name: default for you

WHAT CROSSPLANE ASSUMES

```
# CRD default when providerConfigRef is omitted
# v2 namespaced MR (iam.aws.m.upbound.io/Role)
providerConfigRef:
  kind: ClusterProviderConfig
  name: default
# legacy cluster-scoped MR (iam.aws.upbound.io)
providerConfigRef: {name: default}
```

WHAT THE PLATFORM INSTALLS

```
# one per cluster - same name, different account
apiVersion: aws.m.upbound.io/v1beta1
kind: ClusterProviderConfig
metadata:
  name: default
spec:
  credentials: {source: IRSA}
```

WHY DEFAULT WINS

Less boilerplate

Developers never write it: the XRD defaults providerConfigRef to ClusterProviderConfig / default and the Composition passes it on.

Portable

Same Composition on every cluster – each cluster's default points at its own account.

No 'not found' typos

You can't misspell a name you never write. Imported, observed and rendered MRs just work.

Name it explicitly only when you need a second one: cross-account, or a namespaced ProviderConfig per team.

In-cluster render – not helm template

Helm renders once, before AWS exists. Crossplane renders on every reconcile, after AWS answered.

	helm template	function-go-templating
When	✗ once – on a laptop / in Argo	✓ every reconcile – inside the cluster
Inputs	✗ values.yaml	✓ the live XR + observed state of every child
Cloud IDs	✗ unknown – copy ARNs into values	✓ .observed.resources → ARNs, IDs, status
Drift	✗ only on the next sync	✓ continuous – the loop never stops

COMPOSITION STEP 1 · RENDER WITH WHAT AWS REPORTED

```
delims: {left: '[[', right: ']]'}
[[- $observed := .observed.resources ]]
[[- $bucketArn := dig "bucket" "resource"
    "status" "atProvider" "arn" "" $observed ]]
[[- $logsArn := dig "shared-logs" ...
    "manifest" "status" "atProvider" "arn" ]]

[[- if and $bucketArn $logsArn ]]
apiVersion: iam.aws.m.upbound.io/v1beta1
kind: Policy
spec:
  forProvider:
    policy: [[ ... $bucketArn $logsArn ... ]]
[[- end ]]

# Helm only parses {{ }} - it ships [[ ]] as-is
# so Crossplane renders it, in-cluster
```

Helm ships the template, Crossplane renders it. Not known yet? Don't emit it – the next reconcile will.

Observe what you don't own

provider-kubernetes Object + managementPolicies: [Observe] → read status.* from any existing resource

COMPOSED RESOURCE · READ-ONLY WINDOW

```yaml
apiVersion: kubernetes.m.crossplane.io/v1alpha1
kind: Object
metadata:
  annotations:
    gotemplating.fn.crossplane.io/
      composition-resource-name: shared-logs
spec:
  managementPolicies: ["Observe"]
  readiness:
    policy: DeriveFromObject
  forProvider:
    manifest:        # the thing to watch
      apiVersion: s3.aws.m.upbound.io/v1beta1
      kind: Bucket
      metadata:
        name: platform-logs
        namespace: platform
```

HOW THE VALUE FLOWS

1 · Observe

Provider GETs the bucket. Never creates, updates or deletes it – it belongs to the platform team.

2 · Read status

Whole object lands in status.atProvider.manifest → … manifest.status.atProvider.arn

3 · Use it

go-templating reads it from .observed.resources and writes it into the app's IAM Policy.

Gotchas: readiness DeriveFromObject (SuccessfulCreate never fires on Observe) · RBAC needs patch – SSA dry-run runs even for Observe

Dependencies – function-sequencer

Hold a composed resource back until the ones it depends on are Ready

COMPOSITION STEP 3

```
- step: sequence
  functionRef:
    name: function-sequencer
  input:
    apiVersion: sequencer.fn.crossplane.io/v1beta1
    kind: Input
    rules:
    - sequence:
      - role
      - policy
      - role-policy-attachment
```

WHAT YOU SEE

```
$ kubectl events --for myapp/checkout
Delaying creation of resource(s) matching
"role-policy-attachment" because "policy"
is not fully ready (0 of 1)
```

RECONCILE BY RECONCILE

#1

Bucket · Role · Observe Object

Policy + Attachment held back

#2

Policy

Role Ready + both ARNs observed

#3

RolePolicyAttachment

Role + Policy Ready → refs resolve

No 'referenced field was empty' error storm. Deletes too: enableDeletionSequencing creates Usages → reverse order

Ready – function-auto-ready

The XR is Ready only when every composed resource reports Ready=True

COMPOSED RESOURCES → ONE ANSWER

✓ Object | ✓ Bucket | ✓ Role | ✓ Policy | ✓ Attachment → **MyApp Ready=True**

Why a function?

Composition functions don't mark anything Ready on their own. Without auto-ready (or a manual gotemplating…/ready annotation) the XR stays Ready=False forever.

Where in the pipeline

Right after render, before the sequencer – the sequencer decides what to create from the readiness auto-ready just computed.

Ready ≠ Healthy

If an UPDATE fails, the old AWS resource still exists, so the child stays Ready=True – and so does the XR. ArgoCD says Healthy.

```
$ kubectl -n team-payments get myapp checkout
NAME       HEALTHY   ROLE-ARN                                              SYNCED   READY
checkout   False     arn:aws:iam::123456789012:role/team-payments-checkout   True     True
```

Ready tells you it exists – not that your last change worked. That's the next function's job.

ArgoCD health – function-status-transformer

Lift the child's real AWS error onto the XR, so ArgoCD can turn it Degraded

COMPOSITION STEP 4 · MATCH CHILD → SET XR CONDITION

```
- step: status
  functionRef: {name: function-status-transformer}
  input:
    kind: StatusTransformation
    statusConditionHooks:
    - matchers:
      - type: AnyResourceMatchesAnyCondition
        resources: [{name: ".*"}]
        conditions:
        - {type: Synced, status: "False",
           reason: ReconcileError,
           message: "(?P<Error>(?s).+)"}
      setConditions:
      - target: Composite
        condition: {type: ResourcesHealthy,
          status: "False", message: "{{ .Error }}"}
```

ARGOCD LUA HEALTH CHECK

```
# argocd-cm
resource.customizations.health.
  crossplane.best-practice.com_MyApp: |
  if c.type == "ResourcesHealthy" and
     c.status == "False" then
    return {status="Degraded", message=c.message}
```

WHAT ARGOCD SHOWS

Without

♥ Healthy

error hidden on a child Policy

With

✗ Degraded

MalformedPolicyDocument: Actions must be prefaced by a vendor…

```
ResourcesHealthy=False  ReconcileError  update failed: … CreatePolicyVersion … 400 …
MalformedPolicyDocument: Actions/Conditions must be prefaced by a vendor, e.g., iam, sdb, ec2
```

The developer sees the AWS message on the object they deployed – no kubectl describe hunt across 5 children.

Troubleshoot with crossplane render

Same pipeline on your laptop – the real functions in Docker, no cluster, no AWS

xr.yaml
the XR to render

composition.yaml
the pipeline to run

functions.yaml
which function images

xrd.yaml
schema + defaults

RESULTS

```
crossplane render xr.yaml composition.yaml functions.yaml --xrd xrd.yaml -r
```

```
  spec:
    scope: Namespaced
OUTPUT (SUMMARISED) · WHAT RECONCILE #1 WOULD CREATE
    region:       {default: us-east-1}

kind: MyApp    checkout               # + status.conditions Ready=False
kind: Bucket   checkout
kind: Role     checkout
kind: Object   checkout-shared-logs    # Observe-only
Result: Delaying creation of "policy" because "role" is not fully ready (0 of 1)
Result: Delaying creation of "role-policy-attachment" because "role" is not fully ready (0 of 1)
```

Inspect it live – crossplane resource trace

On a live cluster: the whole MyApp tree, and where it is stuck, in one command

COMMAND · OLDER V2 CLIS: CROSSPLANE BETA TRACE

```
crossplane resource trace myapps.crossplane.best-practice.com/checkout -n team-payments
```

HEALTHY

```
NAME                                                SYNCED   READY   STATUS
MyApp/checkout (team-payments)                      True     True    Available
├─ Policy/checkout (team-payments)                  True     True    Available
├─ RolePolicyAttachment/checkout (team-payments)    True     True    Available
├─ Role/checkout (team-payments)                    True     True    Available
├─ Object/checkout-shared-logs (team-payments)      True     True    Available
└─ Bucket/checkout (team-payments)                  True     True    Available
```

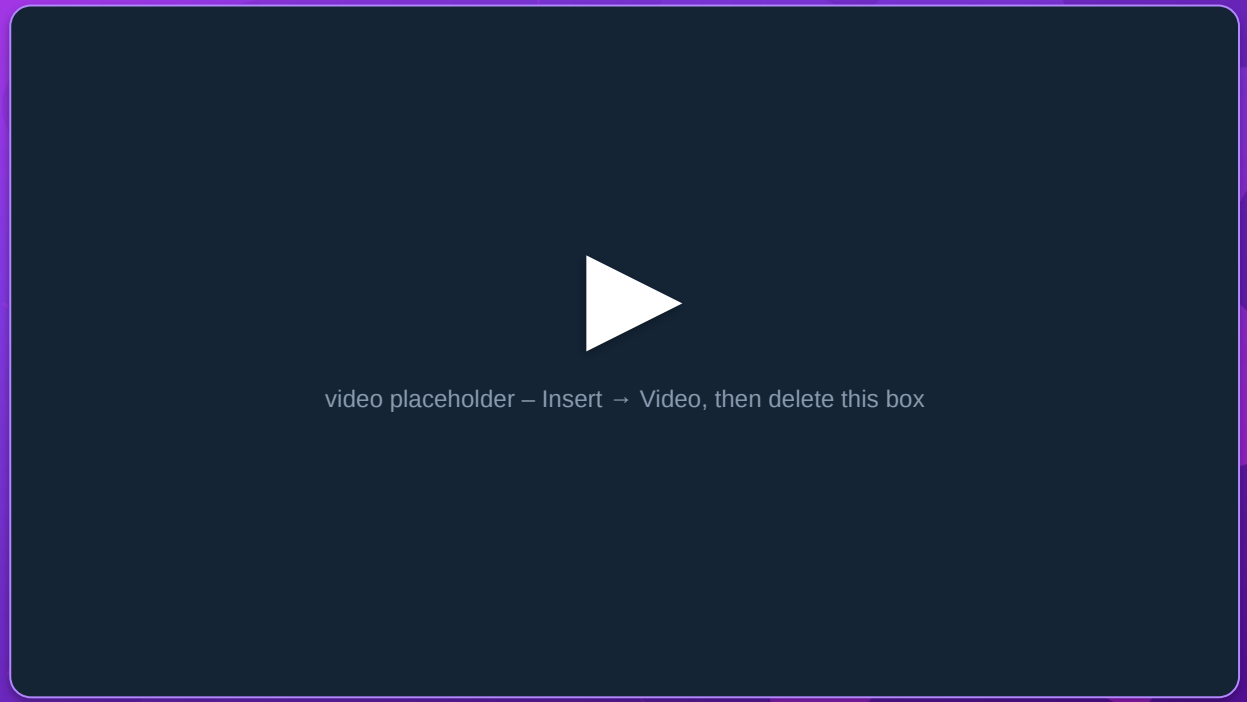
AFTER THE TYPO IN EXTRAPOLICYACTIONS

```
NAME                                                SYNCED   READY   STATUS
MyApp/checkout (team-payments)                      True     True    Available
├─ Policy/checkout (team-payments)                  False    True    ReconcileError: ...ns must be prefaced by a…
├─ RolePolicyAttachment/checkout (team-payments)    True     True    Available
├─ Role/checkout (team-payments)                    True     True    Available
├─ Object/checkout-shared-logs (team-payments)      True     True    Available
└─ Bucket/checkout (team-payments)                  True     True    Available
```



Troubleshooting – live

From a broken Composition to the fix with crossplane render



WATCH FOR

1 · Reproduce

render with the XR from the chart

2 · Replay

-o observed state from the cluster

3 · Read -r

which step dropped / delayed what

4 · Fix & re-render

before you push

Demo: composition

github.com/danielrubin1989/crossplane-composition-best-practices

```
# kind + Crossplane v2.4 + moto (fake AWS)
./demo/demo.sh

# helm install checkout charts/myapp-claim
# helm upgrade ... extraPolicyActions=
#     {s3GetObjectTagging}   <- typo
```

1. Helm installs the platform – the raw [[ ]] is still in the cluster
2. Observe a bucket we don't own → its ARN lands in our IAM Policy
3. Sequencer events, then MyApp Ready only after all 5 children
4. Typo → HEALTHY=False with the AWS message, READY still True
5. crossplane render: create, wire and fail – offline

